

REAL TIME THERMOSYPHON FLOW REVERSAL
PREDICTION THROUGH DATA ASSIMILATION OF COUPLED
COMPUTATIONAL FLUID DYNAMICS AND REDUCED
ORDER MODELING

A Thesis Presented

by

Andrew Reagan

to

The Faculty of the Graduate College

of

The University of Vermont

In Partial Fulfillment of the Requirements
for the Degree of Master of Science
Specializing in Applied Mathematics

January, 2013

Accepted by the Faculty of the Graduate College, The University of Vermont, in partial fulfillment of the requirements for the degree of Master of Science, specializing in Applied Mathematics.

Thesis Examination Committee:

Chris Danforth, Ph.D. Advisor

Peter Dodds, Ph.D.

Yves Dubief, Ph.D. Chairperson

Domenico Grasso, Ph.D. Dean, Graduate College

Date: November 15, 2013

Abstract

A thermal convection loop is a circular chamber filled with water, heated on the bottom half and cooled on the top half. With sufficiently great forcing of heat, the direction of fluid flow in the loop oscillates chaotically, thus forming an analog to the real weather system. Like full scale weather models, we can only observe a small part of state space, making prediction very difficult. To overcome this challenge, data assimilation methods and specifically ensemble methods use the computational model itself to estimate the uncertainty of our model to optimally combine these observations into an initial condition for predicting the future. First, we build and verify four distinct DA methods. Then, a computational fluid dynamic simulation of the loop and a reduced order model are both used by these DA methods to predict flow reversals. This happens in real time as experimental data is fed into the algorithm.

in dedication to

my parents, Donna and Kevin Reagan, for their unwavering support

Acknowledgements

I would like to thank professors Chris Danforth and Peter Dodds for their outstanding advising. I would like to thank my fellow students for all of their help and toleration through the past two years. And finally I would like to thank my girlfriend Sam Spisiak for knowing more than anyone else about this thesis.

Table of Contents

Dedication	ii
Acknowledgements	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Introduction	1
1.2 History of NWP	2
1.3 IC determination: data assimilation	6
1.4 NWP and fluid dynamics: computational models	6
1.5 Toy climate experiments	7
2 Data Assimilation	8
2.0.1 3D-Var	8
2.0.2 Extended Kalman Filter	8
2.0.3 Ensemble Kalman Filter	9
2.1 Coding the TLM	12
3 Computational Fluid Dynamics	15
3.1 Derivation of Governing Equations	15
3.2 Discretization and Solving Algorithms	15
3.3 Solving with OpenFOAM	16
4 Thermal Convection Loop Experiment	18
4.1 Thermal Convection Loops	18
4.2 Detailed Experimental Setup	18
5 Results	20
5.1 Prediction Skill for Synthetic Data	20
5.1.1 Sampling frequency, distribution	20
5.1.2 Random error vs PMD perturbation scheme	20
5.1.3 Maintaining hydrostatic balance	21
5.2 Real-time Flow Reversal Prediction Skill	21

6 Conclusion	22
6.1 Main Result	22
6.1.1 More Details	22
Bibliography	23
Appendices	25
A Parameters	25
B Code	26

List of Figures

2.1	A sample timeseries of the ensembles used in the EnKF. In all tests, as seen here, 10 ensemble members are used.	10
2.2	The RMS error (unscaled by climatology) is reported for our EKF and EnKF filters, measured as the difference between forecast and truth at the end of an assimilation window for the latter 2500 assimilation windows in a 3000 assimilation window model run. Error is measured in the only observed variable, x_1 . Increasing the assimilation window led to an decrease in predictive skill, as expected. Additive and multiplicative covariance inflation is the same as Harris et. al., I've ran a few experiments to tune these, I should have my own tuning for every window length soon.	10
2.3	For near-perfect initial knowledge of the atmospheric state, the RMS error of a prediction using DA is compared against a prediction that does not. Initial error is normal with mean 0 and variance 0.01, and the RMS errors reported are averages of 100 runs. Observational noise is normally distributed with mean 0 and variance 0.5, and an assimilation window length of 0.261 is used (approximately 5% of climatological variation). As would be expected, in the long run DA greatly improves forecast skill, as forecasts without DA saturate to climatological error. The initial period where a forecast exceeds the skill of DA is potentially due to the spin-up time required by the filter to dial in the analysis error covariance.	11
2.4	The difference of ensemble forecasts from the analysis is reported for 760 assimilation windows in one model run of length 200, with 10 ensemble members and an assimilation window of length 0.261. This has the same shape of as the difference between ensemble forecasts and the mean of the forecasts (I have this as well). This spread of ensemble forecasts is what allows us to estimate the error covariance of the forecast model, and appears fall from a normal distribution.	12
2.5	An explanation of how and why the best way to obtain a TLM is with a differentiated numerical scheme.	13
3.1	A snapshot of the mesh used for CFD simulations.	16
3.2	A snapshot of the mesh used for CFD simulations.	17
4.1	Professor Chris Danforth with the aluminum rig.	19

List of Tables

2.1	Summary of Lorenz '63 parameters	14
A.1	EKF Covariance Inflation	25

Chapter 1

Introduction

In this chapter we explore the current state of numerical weather prediction, in particular data assimilation, computational fluid dynamics, and reduced order experiments.

1.1 Introduction

Prediction of the future state of many complex systems is integral to the functioning of our society. Some of these systems include weather (Hsiang et al. 2013), health (Ginsberg et al. 2008), the economy (Bollen et al. 2011, Sornette and Zhou 2006), marketing (Asur and Huberman 2010) and engineering (Savely et al. 1972). For weather in particular, this prediction happens on supercomputers across the world, as numerical weather model integrations taking our current best guess of the weather into the future. The accuracy of these predictions depend on the accuracy of the models themselves, and the quality of our current knowledge of the atmosphere.

Model accuracy has improved with better meteorological understanding of weather processes and advances in computing technology. To solve the initial value problem, techniques developed over the past 50 years are now broadly known as data assimilation. Formally, data assimilation is the process of using all available information, including short-range model forecasts and observations, to estimate the current state of a system as accurately as possible (Yang et al. 2006).

CHAPTER 1. INTRODUCTION

We employ a toy climate experiment as a testbed for improving numerical weather prediction algorithms, namely data assimilation methods. This approach is akin to the historical development of current methodologies, and provides a tractable system for rigorous analysis. The experiment is a thermal convection loop, which by design simplifies our problem into the prediction of mostly convection. The dynamics of thermal convection loops have been explored under both periodic (Keller) and chaotic (Welander 1995, Creveling et al. 1975, Gorman and Widmann 1984, Gorman et al. 1986, Ehrhard and Müller 1990, Yuen and Bau 1999, Jiang and Shoji 2003, Burroughs et al. 2005, Desrayaud et al. 2006, Yang et al. 2006, Ridouane et al. 2010) regimes. A full characterization of the computational behavior of a loop under flux boundary conditions by Louissos et. al. delimited four regimes: chaotic convection with reversals, high Ra aperiodic stable convection, steady stable convection, and conduction/quasi-conduction (Louissos et al. 2013). For the remainder of this work, we focus on the chaotic flow regime.

Computational simulations of the thermal convection loop are performed with the open-source finite volume C++ library OpenFOAM. The open-source nature of this software enable its integration with the data assimilation algorithms that were verified with a reduced order model.

1.2 History of NWP

The importance of Bjerknes in early developments in NWP is clearly described by Thompson’s “Charney and the revival of NWP” (Thompson 1990):

It was not until 1904 that Vilhelm Bjerknes - in a remarkable manifesto and testament of deterministic faith - started the central problem of NWP. This was the first explicit, coherent, recognition that the future state of the atmosphere is, *in principle*, completely determined by its detailed initial state and known boundary conditions, together with Newton’s equations of motion, the Boyle-Charles-Dalton equation of state, the equation of mass continuity, and the thermodynamic energy equation. Bjerknes went further: he outlined an ambitious, but logical program of observation, graphical analysis of meteorological data and graphical solution of the governing equations. He succeeded in persuading the Norwegians to support an expanded network of surface observation stations, founded the famous Bergen School of synoptic and dynamic meteorology, and ushered in the famous polar front theory of cyclone formation. Beyond providing a clear

CHAPTER 1. INTRODUCTION

goal and a sound physical approach to dynamical weather prediction, V. Bjerknes instilled his ideas in the minds of his students and their students in Bergen and in Oslo, three of whom were later to write important chapters in the development of NWP in the US (Rossby, Eliassen, and Fjortoft).

It was then until 1922 that Richardson suggested a practical way to solve these equations. From Jule G. Charney's "Dynamical forecasting by numerical process" (?) we recognize Richardson's achievement:

He [Richardson] proposed to integrate the equations of motion numerically and showed exactly how this might be done. That the actual forecast used to test his method was unsuccessful was in no way a measure of the value of his work. In retrospect it becomes obvious that the inadequacies of observation alone would have doomed any attempt, however well conceived, a circumstance of which Richardson was aware. The real value of his work lay in the fact that it crystallized once and for all the essential problems that would have to be faced by future workers in the field and it laid down a thorough groundwork for their solution.

For a long time no one ventured to follow in Richardson's footsteps. The paucity of the observational network and the enormity of the computational task stood as apparently insurmountable barriers to the realization of his dream that one day it might be possible to advance the computation faster than the weather. But with the increase in the density and extent of the surface and upper-air observational network on the one hand, and the development of large-capacity high-speed computing machines on the other, interest has revived in Richardson's problem, and attempts have been made to attack it anew.

These efforts have been characterized by a devotion to objectives more limited than Richardson's. Instead of attempting to deal with the atmosphere in all its complexity, one tries to be satisfied with simplified models approximating the actual motions to a greater or lesser degree. By starting with models incorporating only what is thought to be the most important of the atmospheric influences, and by gradually bringing in the others, one is able to proceed inductively and thereby avoid the pitfalls inevitably encountered when a great many poorly understood factors are introduced all at once.

CHAPTER 1. INTRODUCTION

A necessary condition for the success of this stepwise method is, of course, that the first approximations bear a recognizable resemblance to the actual motions. Fortunately, the science of meteorology has progressed to the point where one feels that at least the main factors governing the large-scale atmospheric motion are well known. Thus integrations of even the linearized barotropic and thermally inactive baroclinic equations have yielded solutions bearing a marked resemblance to reality. At any rate, it seems clear that the models embodying the collective experience and the positive skill of the forecast cannot fail utterly. This conviction has served as the guiding principle in the work of the meteorology project at The Institute for Advanced Study [at Princeton University] with which the writer has been connected.

Richardson used a horizontal grid of about 200km, and 4 vertical layers in his mesh of Germany. Using the observations available, he computed the time derivative of pressure in the central box, predicting a change of 146 hPa whereas in reality there was almost no change. This huge discrepancy was due mostly to the fact that his initial conditions were not balanced, and therefore included fast-moving gravity waves which masked the true climate signal. If the integration had continued, it would have “exploded,” due to the Courant number induced by his mesh and timestep, which was greater than 1. We will revisit this condition in Chapter 3, but a simple explanation is as follows. The Courant number relates the movement of a quantity to the size of the mesh, written as

$$C = \frac{u_x \Delta t}{\Delta x}$$

in one dimension, and a value > 1 for a finite difference method solving partial differential conditions is divergent.

Early on, the problem of fast travelling waves from unbalanced IC was solved by filtering the equations of motion based on the quasi-geostrophic (slow time) balance. Although this made running models feasible on the computers of WW2, the incorporation of the full equations proved necessary for more accurate forecasts to be made.

The first computer model forecast was performed as a one day forecast on the ENIAC (the Electronic Numerical Integrator and Computer) by none other than J. von Neuman (?), at the Aberdeen US Army

CHAPTER 1. INTRODUCTION

facility in Maryland. This incorporated von Neumann's "stored programming" technique (the ability to write "for" loops) and produced a forecast that remaining at least plausible.

Charney already saw the success of models with filtered equations of motion would not have enough accuracy and understood the difficulties that would be overcome in the next 50 years:

The discussion so far has dealt exclusively with the quasi-geostrophic equations as the basis for numerical forecasting. Yet there has been no intention to exclude the possibility that the primitive Eulerian equations can also be used for this purpose. The outlook for numerical forecasting would be indeed dismal if the quasi-geostrophic approximation represented the upper limit of attainable accuracy, for it is known that it applies only indifferently, if at all, to many of the small-scale but meteorologically significant motions. We have merely indicated two obstacles that stand in the way of the applications of the primitive equations: first, there is the difficulty raised by Richardson that the horizontal divergence cannot be measured with significant accuracy. Moreover, the horizontal divergence is only one of a class of meteorological unobservables which also includes the horizontal acceleration. And second, if the primitive Eulerian equations are employed, a stringent and seemingly artificial bound is imposed on the size of the time interval for the finite difference equations. The first obstacle is the most formidable, for the second only means that the integration must proceed in the steps of the order of fifteen minutes rather than two hours. Yet the first does not seem insurmountable, as the following considerations will indicate.

What he describes is an integration where the initial fast-moving waves subside through the geostrophic adjustment of the model after a day, and do not otherwise affect the forecast. Charney concludes his 1950 paper with the following discussion of a problem that is still an active area of research, resolving subgrid-scale phenomena:

Nonadiabatic and frictional terms have been ignored in the body of the discussion because it was thought that one should first seek to determine how much of the motion could be explained without them. Ultimately they will have to be taken into account, particularly if the forecast period is to be extended to three or more days.

Condensation phenomena appear to be the simplest to introduce: one has only to add the equation of continuity for water vapor and replace the dry air by the moist adiabatic equation.

CHAPTER 1. INTRODUCTION

Long-wave radiational effects can also be provided for, since our knowledge of the absorptive properties of water vapor and carbon dioxide has progressed to a point where quantitative estimates of radiational cooling can be made, although the presence of clouds will complicate the problem considerably.

The most difficult phenomena to include have to do with the turbulent transfer of momentum and heat. A great deal of research remains to be done before enough is known about these effects to permit the assignment of even rough values to the eddy coefficients of viscosity and heat conduction. Owing to their statistically indeterminate nature, the turbulent properties of the atmosphere place an upper limit to the accuracy obtainable by dynamical methods of forecasting, beyond which we shall have to rely upon statistical methods. But it seems certain that much progress can be made before these limits can be reached.

The shortcomings of our understanding of the subgrid-scale phenomena do impose a limit on our ability to predict the weather, but this is not the upper limit. The upper limit of predictability exists for even a perfect model, as E. Lorenz shows in (Lorenz 1963) with a three variable model that exhibits sensitive dependence on initial conditions.

1.3 IC determination: data assimilation

The purpose of data assimilation is defined by Talagrand (Talagrand 1997) as “using all the available information, to determine as accurately as possible the state of the atmospheric (or oceanic) flow.” Broadly, data assimilation algorithms are used from areas as disparate as quadcopter stabilization (Achtelik et al. 2009) to finance (?).

Describe statistical problem, OI 3D-VAR EKF EnKF 4D-VAR

1.4 NWP and fluid dynamics: computational models

Derived from Newton’s law of motion, the basic equations governing flow of a viscous, heat-conducting fluid have been known since as early as 1823 (Navier 1823) and in their current form since 1846 (Stokes 1846). The principle equations is the momentum equation, supplemented by the continuity equation and the energy

CHAPTER 1. INTRODUCTION

equation. These three equations are given by

$$\begin{aligned}\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j} [\rho u_j] &= 0 \\ \frac{\partial}{\partial t} (\rho u_i) + \frac{\partial}{\partial x_j} [\rho u_i u_j + p \delta_{ij} - \tau_{ji}] &= 0, \quad i = 1, 2, 3 \\ \frac{\partial}{\partial t} (\rho e_0) + \frac{\partial}{\partial x_j} [\rho u_j e_0 + u_j p + q_j - u_i \tau_{ij}] &= 0\end{aligned}$$

1.5 Toy climate experiments

Thermal convection loops are one of the few known experimental implementations of Lorenz's 1963 system of equations. The convection loop is a hula-hoop shaped tube of water, heated on the bottom by a fixed temperature water bath and cooled on the top by room temperature circulating air. Varying the forcing of temperature applied to the bottom half of the loop there are three experimentally accessible flow regimes of interest to us: conduction, steady convection, and convection of oscillating directions. The latter regime exhibits the much studied phenomenon of chaos: the approximate past does not approximately determine the future.

Chapter 2

Data Assimilation

Specifically, I test a MATLAB implementation of the Extended Kalman Filter (EKF) and Ensemble Kalman Filter (EnKF) on the Lorenz 1963 three-variable model:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= rx - y - xz \\ \frac{dz}{dt} &= xy - bz\end{aligned}$$

The canonical choice of $\sigma = 10$, $b = 8/3$ and $r = 28$ is used for all tests here, which produce the well known butterfly attractor (Figure ??).

2.0.1 3D-Var

Most operational weather centers today operate with a 3D-var filter.

2.0.2 Extended Kalman Filter

For starters, I coded an EKF. The EKF is the “gold standard” of data assimilation methods. It works by updating our knowledge of error covariance matrix for each assimilation window, using a Tangent Linear Model (TLM). Using the notation of Kalnay (Kalnay 2003), this amounts to making a forecast with the nonlinear model M , and updating the error covariance matrix P with the TLM L , and adjoint model L^T

$$x^f(t_i) = M_{i-1}[x^a(t_{i-1})]$$

$$P^f(t_i) = L_{i-1}P^a(t_{i-1})L_{i-1}^T + Q(t_{i-1})$$

where Q is the noise covariance matrix (model error). Here $Q = 0$ since our model is perfect.

The analysis step is then written as (for H the observation operator):

$$x^a(t_i) = x^f(t_i) + K_i d_i$$

$$P^a(t_i) = (I - K_i H_i) P^f(t_i)$$

where

$$d_i = y_i^o - H[x^f(t_i)]$$

is the innovation.

The Kalman gain matrix is computed to minimize the analysis error covariance P_i^a as

$$K_i = P^f(t_i) H_i^T [R_i + H_i P^f(t_i) H_i^T]^{-1}$$

where R_i is the observation error covariance.

Since we are making observations of the truth with random normal errors of standard deviation ϵ , then R_i is a matrix of just ϵ values.

The most difficult, and most computationally expensive, part of the EKF is deriving and integrating the TLM. For this reason, no operational weather centers use the EKF, although approximations such as the EnKF are promising (next).

2.0.3 Ensemble Kalman Filter

The computational cost of the EKF is mitigated through the approximation of the error covariance matrix C_f from an ensemble of predictions.

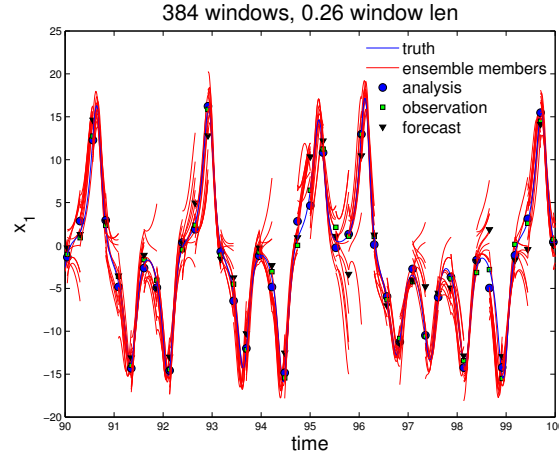


Figure 2.1: A sample timeseries of the ensembles used in the EnKF. In all tests, as seen here, 10 ensemble members are used.

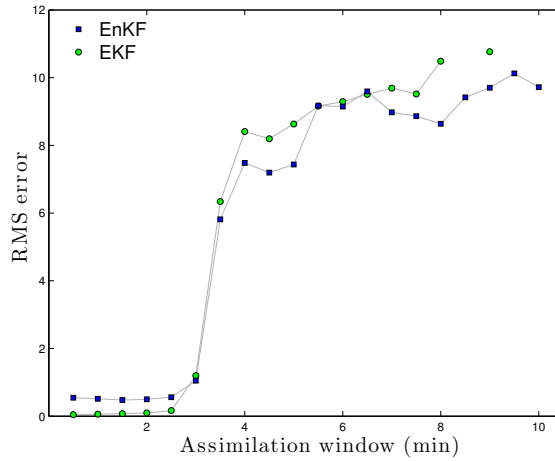


Figure 2.2: The RMS error (unscaled by climatology) is reported for our EKF and EnKF filters, measured as the difference between forecast and truth at the end of an assimilation window for the latter 2500 assimilation windows in a 3000 assimilation window model run. Error is measured in the only observed variable, x_1 . Increasing the assimilation window led to an decrease in predictive skill, as expected. Additive and multiplicative covariance inflation is the same as Harris et. al., I've ran a few experiments to tune these, I should have my own tuning for every window length soon.

As an alternative to the prohibitively costly EKF, filters which approximate the error covariance matrix from other methods are gaining wind. One such approach is the use of a forecast ensemble, where a collection of models (ensemble members) are initialized from the prior analysis x^a with random error. The spread of these models with randoms errors is then used to estimate the error covariance.

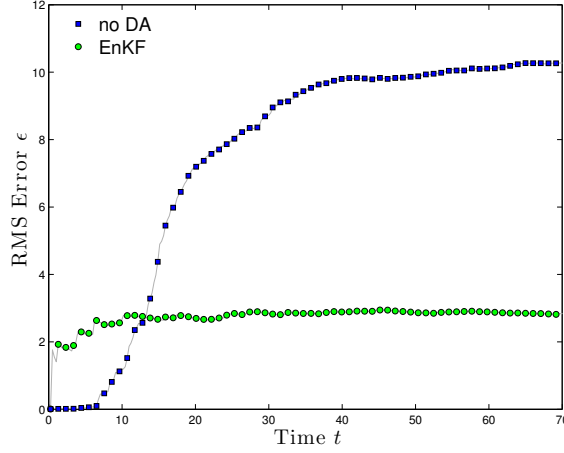


Figure 2.3: For near-perfect initial knowledge of the atmospheric state, the RMS error of a prediction using DA is compared against a prediction that does not. Initial error is normal with mean 0 and variance 0.01, and the RMS errors reported are averages of 100 runs. Observational noise is normally distributed with mean 0 and variance 0.5, and an assimilation window length of 0.261 is used (approximately 5% of climatological variation). As would be expected, in the long run DA greatly improves forecast skill, as forecasts without DA saturate to climatological error. The initial period where a forecast exceeds the skill of DA is potentially due to the spin-up time required by the filter to dial in the analysis error covariance.

Since this method is only an approximation, it often fails to capture the full spread of error, and so additive and multiplicative inflation factors are used to obtain a good estimate of the error covariance.

The only difference between this approach and the EKF, numerically, is that the forecast error covariance P^f is computed from the ensemble members, without the need for a tangent linear model:

$$P^f \approx \frac{1}{K-2} \sum_{k \neq l} \left(x_k^f - \bar{x}_l^f \right) \left(x_k^f - \bar{x}_l^f \right)^T.$$

In computing the error covariance P from the ensemble, we wish to add up the error covariance of each forecast with respect to the mean forecast. But this would underestimate the error covariance since the forecast we're comparing was used in the ensemble average (to obtain the mean forecast). So here to computing the error covariance matrix for each forecast, that forecast itself is excluded from the ensemble average forecast.

We can see the classic spaghetti of the ensemble with this filter implemented on Lorenz 63 in Figure 2.1.

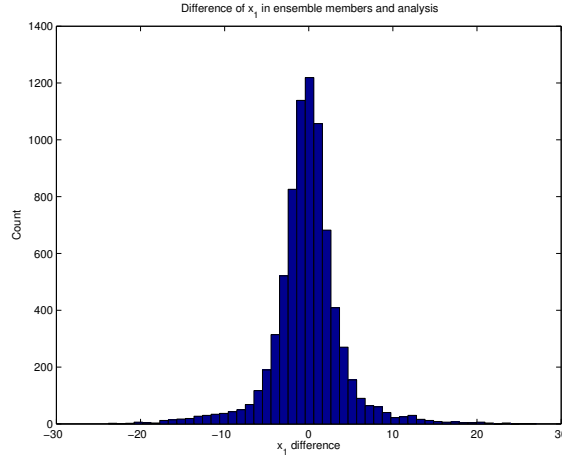


Figure 2.4: The difference of ensemble forecasts from the analysis is reported for 760 assimilation windows in one model run of length 200, with 10 ensemble members and an assimilation window of length 0.261. This has the same shape of as the difference between ensemble forecasts and the mean of the forecasts (I have this as well). This spread of ensemble forecasts is what allows us to estimate the error covariance of the forecast model, and appears fall from a normal distribution.

2.1 Coding the TLM

Actually coding the TLM was difficult, so I'll try to spell it out here, to make sure I understand it. The TLM is the model which advances an initial perturbation δx_i at timestep i to a final perturbation $\delta^* x_{i+1}$ at timestep $i + 1$. The dynamical system we are interested in, Lorenz '63, is given as a system of ODE's:

$$\frac{dx}{dt} = F(x).$$

We integrate this system using a numerical scheme of our choice (here I used RK2), to obtain a model M discretized in time.

$$x(t) = M[x(t_0)].$$

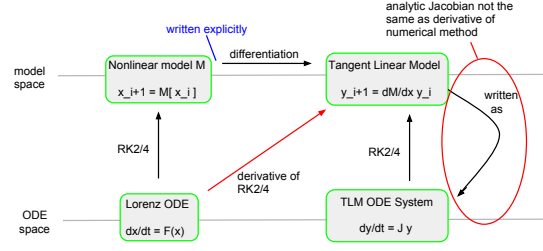


Figure 2.5: An explanation of how and why the best way to obtain a TLM is with a differentiated numerical scheme.

Introducing a small perturbation y , we can approximate the our model M of $x(t_0) + y(t_0)$ with a Taylor series around $x(t_0)$, for which we assume to already have solved for:

$$\begin{aligned} M[x(t_0) + y(t_0)] &= M[x(t_0)] + \frac{\partial M}{\partial x} y(t_0) + O[y(t_0)^2] \\ &\approx x(t) + \frac{\partial M}{\partial x} y(t_0). \end{aligned}$$

We can then solve for the linear evolution of the small perturbation $y(t_0)$ as

$$\frac{dy}{dt} = Jy$$

where $J = \partial F / \partial x$ is the Jacobian of F . We can solve the above system of linear ordinary differential equations using the same numerical scheme as we did for the nonlinear model.

The problem with solving that system of equations is that the Jacobian matrix of discretized code is not necessarily identical to the discretization of the Jacobian operator for the analytic system. This is a problem because we need to have the TLM of our model M , which is the time-space discretization of the solution to $dx/dt = F(x)$. We can apply our numerical method to the $dx/dt = F(x)$ to obtain M explicitly, and then take the Jacobian of this. But, that is extremely messy (and costly), since recall the RK4 method had many nested function iterations. It is therefore desirable to take the derivative of the numerical scheme directly, and apply this differentiated numerical scheme to the system of equations $F(x)$ to obtain the TLM. A schematic of this scenario is illustrated in Figure 2.5. It is for this reason that there exists code that takes the derivative of numerical code (i.e. Fortran) for implementing the EKF on models larger than 3 dimensions with numerical schemes more complicated than RK2 or RK4.

Table 2.1: Summary of Lorenz '63 parameters

Parameter	Value	Interpretation, if any
σ	10	
b	$8/3$	
r	28	

The canonical choice of $\sigma = 10$, $b = 8/3$ and $r = 28$ is used for all tests here, which produce the well known butterfly attractor (Figure ??).

Chapter 3

Computational Fluid Dynamics

3.1 Derivation of Governing Equations

Closely following the original derivation of Stokes (Stokes 1846), I present a derivation of the main equations governing the flow of water at temperatures near 300K and Rayleigh number within reason.

Follow: <http://www.allstar.fiu.edu/aero/Flow2.htm> and http://www.cfd-online.com/Wiki/Navier-Stokes_equations

3.2 Discretization and Solving Algorithms

A spatial and temporal discretization of the governing equations is necessary for their solution numerically.

Specifically, a combined SIMPLE/PISO algorithm is used. The PISO (Pressure-Implicit with Splitting of Operators) algorithm derives from the work of (Issa 1986), and is complementary to the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) (Patankar and Spalding 1972) iterative method.

Simple algorithm: http://www.cfd-online.com/Wiki/SIMPLE_algorithm

The proper inclusion of turbulent effects with calculated Raleigh number are important, and are modeled using LES (Large Eddy Simulation).

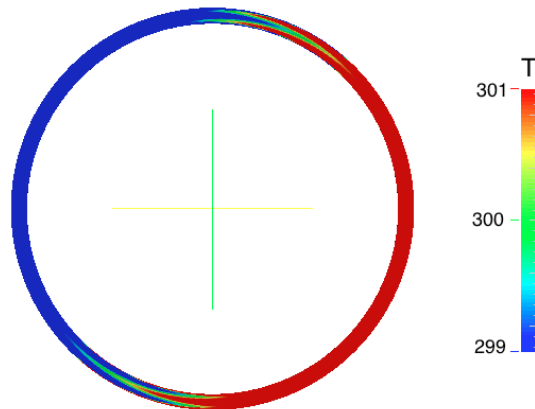


Figure 3.1: A snapshot of the mesh used for CFD simulations.

3.3 Solving with OpenFOAM

Using the aforementioned numerical schemes, I perform a computational fluid dynamic simulation of the entire thermosyphon using the open-source software OpenFOAM. First, we begin by creating 2 and 3-dimensional meshes of the thermosyphon natively in OpenFOAM's blockMesh utility. To reduce computational bandwidth, it was important to first create the mesh and then renumber using the renumberMesh utility. The 2D mesh contains 36000 points, and is imposed with fixed flux boundary conditions on the top and bottom of the loop. This fixed flux boundary condition is implemented through the externalWallHeatFluxTemperature library, with some difficulty. The 3D mesh contains over 1M points, and we have chosen to explore the 2D model for now.

Links for OpenFOAM code documentation:

<http://www.openfoam.org/docs/user/compiling-applications.php>

<http://www.openfoam.org/docs/cpp/>

Here is a picture of the thermosyphon colored by temperature.

And here is the 2D mesh:

The solver used is buoyantBoussinesqPimpleFoam, which I'll now explain.

The Boussinesq approximation makes the assumption that density changes due to temperature differences are sufficiently small so that they can be neglected without affecting the flow, except when multiplied by

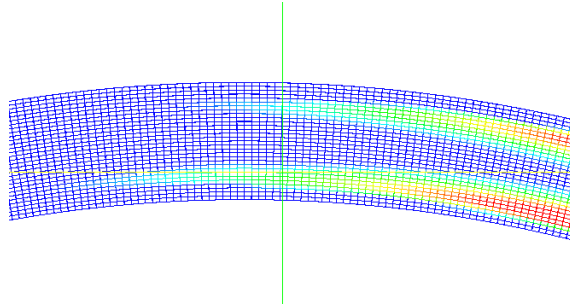


Figure 3.2: A snapshot of the mesh used for CFD simulations.

gravity. The non-dimensionalized density difference $\Delta\rho/\rho$ depends on the value of heat flux into the loop, but remains small even for large values of flux. (derive an equation for this ratio in terms of flux q'').

Chapter 4

Thermal Convection Loop Experiment

The thermosyphon, a type of natural convection loop or non-mechanical heat pump, can be likened to a toy model of climate (Harris et al. 2011).

4.1 Thermal Convection Loops

Under certain conditions, thermal convection loops can operate as non-mechanical heat pumps, making them useful in many applications. Thermosyphons are used in solar hot water heaters (Belessiotis and Mathioulakis 2002), cooling systems for computers (Beitelmal and Patel 2002), roads and railways that cross permafrost (Cherry et al. 2006), nuclear power plants (Detman 1968, Beine et al. 1992, Kwant and Boardman 1992), and other industrial applications.

4.2 Detailed Experimental Setup

Originally built (and subsequently melted) by Dr. Chris Danforth, the University of Vermont's current experimental thermosyphons have made considerable progress. Operated by Dave Hammond, a Scientific Electronics Technician and master parameter tuner, the experimental thermosyphons access the chaotic regime of state space found in the principled governing equations.

CHAPTER 4. THERMOSYPHON



Figure 4.1: Professor Chris Danforth with the aluminum rig.

Chapter 5

Results

5.1 Prediction Skill for Synthetic Data

I first present the results pertaining to the accuracy of forecasts for synthetic data.

5.1.1 Sampling frequency, distribution

Increasing sampling frequency led to improved forecast accuracy, up to a finite cutoff where the error covariance inflation requirement becomes unstable.

Observational network density also improves forecast accuracy, as would be expected, up to limit of the computational memory available for computing large non-sparse covariance matrices. For example the full system, with 216,000 variables, produces an error covariance matrix of 347GB in double precision. With the expected continuation of Moore's Law, however, this simulation may be possible in a few as 5 years (assuming a doubling of memory every 18 months, starting now with my 8GB laptop) on a laptop computer.

5.1.2 Random error vs PMD perturbation scheme

The computational implementation of the NS equations is dispersive, meaning that small perturbations are quickly resolved.

In other words, we beat the hell of out random error.

CHAPTER 5.

5.1.3 Maintaining hydrostatic balance

The importance of maintaining balance in the model variables, originally dooming the efforts of Richardson, was taken into consideration.

5.2 Real-time Flow Reversal Prediction Skill

Now the real business: running CFD in real time and assimilating observations from temperature probes.

Chapter 6

Conclusion

6.1 Main Result

Here is a different citation (Bongard and Paul 2000).

6.1.1 More Details

And one more (Auerbach and Bongard 2010).

BIBLIOGRAPHY

Bibliography

- Achtelik, M., T. Zhang, K. Kuhnlenz, and M. Buss (2009). Visual tracking and control of a quadcopter using a stereo camera system and inertial sensors. In *Mechatronics and Automation, 2009. ICMA 2009. International Conference on*, pp. 2863–2869. IEEE.
- Asur, S. and B. A. Huberman (2010). Predicting the future with social media. In *Web Intelligence and Intelligent Agent Technology (WI-IAT), 2010 IEEE/WIC/ACM International Conference on*, Volume 1, pp. 492–499. IEEE.
- Auerbach, J. E. and J. C. Bongard (2010). Evolving CPPNs to grow three-dimensional physical structures. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2010)*, New York, NY, pp. 627–634. ACM.
- Beine, B., V. Kaminski, and W. Von Lensa (1992). Integrated design of prestressed cast-iron pressure vessel and passive heat removal system for the reactor cell of a 200 mwth modular reactor. *Nuclear engineering and design* 136(1), 135–141.
- Beitelmal, M. H. and C. D. Patel (2002). Two-phase loop: Compact thermosyphon. *Approved for External Publication*.
- Belessiotis, V. and E. Mathioulakis (2002). Analytical approach of thermosyphon solar domestic hot water system performance. *Solar Energy* 72(4), 307–315.
- Bollen, J., H. Mao, and X. Zeng (2011). Twitter mood predicts the stock market. *Journal of Computational Science* 2(1), 1–8.
- Bongard, J. C. and C. Paul (2000). Investigating morphological symmetry and locomotive efficiency using virtual embodied evolution. In *From Animals to Animats: The Sixth International Conference on the Simulation of Adaptive Behaviour*, pp. 420–429. MIT Press.
- Burroughs, E., E. Coutsias, and L. Romero (2005). A reduced-order partial differential equation model for the flow in a thermosyphon. *Journal of Fluid Mechanics* 543, 203–238.
- Cherry, B., Z. Dan, and A. Lustgarten (2006). Next stop, lhasa. *Fortune* 153, 154.
- Creveling, H., D. PAZ, J. Baladi, and R. Schoenhals (1975). Stability characteristics of a single-phase free convection loop. *Journal of Fluid Mechanics* 67(part 1), 65–84.
- Desrayaud, G., A. Fichera, and M. Marcoux (2006). Numerical investigation of natural circulation in a 2d-annular closed-loop thermosyphon. *International journal of heat and fluid flow* 27(1), 154–166.
- Detman, R. F. (1968, July 16). Thermos iphon depp pool reactor. US Patent 3,393,127.
- Ehrhard, P. and U. Müller (1990). Dynamical behaviour of natural convection in a single-phase loop. *Journal of Fluid mechanics* 217, 487–518.
- Ginsberg, J., M. H. Mohebbi, R. S. Patel, L. Brammer, M. S. Smolinski, and L. Brilliant (2008). Detecting influenza epidemics using search engine query data. *Nature* 457(7232), 1012–1014.
- Gorman, M. and P. Widmann (1984). Chaotic flow regimes in a convection loop. *Phys. Rev. Lett.* (52), 2241–2244.
- Gorman, M., P. Widmann, and K. Robbins (1986). Nonlinear dynamics of a convection loop: a quantitative comparison of experiment with theory. *Physica D* (19), 255–267.
- Harris, K. D., E. H. Ridouane, D. L. Hitt, and C. M. Danforth (2011). Predicting flow reversals in chaotic natural convection using data assimilation. *arXiv preprint arXiv:1108.5685*.
- Hsiang, S. M., M. Burke, and E. Miguel (2013). Quantifying the influence of climate on human conflict. *Science* 341(6151).

BIBLIOGRAPHY

- Issa, R. I. (1986). Solution of the implicitly discretised fluid flow equations by operator-splitting. *Journal of Computational physics* 62(1), 40–65.
- Jiang, Y. and M. Shoji (2003). Spatial and temporal stabilities of flow in a natural circulation loop: influences of thermal boundary condition. *J Heat Trans.* (125), 612–623.
- Kalnay, E. (2003). *Atmospheric modeling, data assimilation, and predictability*. Cambridge university press.
- Keller, J. Periodic oscillations in a model of thermal convection.
- Kwant, W. and C. Boardman (1992). Prismliquid metal cooled reactor plant design and performance. *Nuclear engineering and design* 136(1), 111–120.
- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the atmospheric sciences* 20(2), 130–141.
- Louisos, W. F., D. L. Hitt, and C. M. Danforth (2013). Chaotic flow in a 2d natural convection loop with heat flux boundaries. *International Journal of Heat and Mass Transfer* 61, 565–576.
- Navier, C. (1823). Mémoire sur les lois du mouvement des fluides. *Mémoires de l'Académie Royale des Sciences de l'Institut de France* 6, 389–440.
- Patankar, S. V. and D. B. Spalding (1972). A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer* 15(10), 1787–1806.
- Ridouane, E. H., C. M. Danforth, and D. L. Hitt (2010). A 2-d numerical study of chaotic flow in a natural convection loop. *International Journal of Heat and Mass Transfer* 53(1), 76–84.
- Savely, R., B. Cockrell, , and S. Pines (1972). Apollo experience report - onboard navigational and alignment software. *Technical Report*.
- Sornette, D. and W.-X. Zhou (2006). Predictability of large future changes in major financial indices. *International Journal of Forecasting* 22(1), 153–168.
- Stokes, S. G. G. (1846). *On the Friction of Fluids in Motion, and the Equilibrium and Motion of Elastic Solids*.
- Talagrand, O. (1997). Assimilation of observations, an introduction. *JOURNAL-METEOROLOGICAL SOCIETY OF JAPAN SERIES 2* 75, 81–99.
- Thompson, P. (1990). Charney and the revival of nwp. *The Atmosphere - a challenge*, 93–119.
- Welander, P. (1995). On the oscillatory instability of a differentially heated fluid loop. *International Geophysics Series* 59.
- Yang, S.-C., D. Baker, H. Li, K. Cordes, M. Huff, G. Nagpal, E. Okereke, J. Villafane, E. Kalnay, and G. S. Duane (2006). Data assimilation as synchronization of truth and model: Experiments with the three-variable lorenz system*. *Journal of the atmospheric sciences* 63(9), 2340–2354.
- Yuen, P. and H. Bau (1999). Optimal and adaptive control of chaotic convection. *Phys. Fluids* (11), 1435–1448.

Appendix A: Parameters

Table A.1: EKF Covariance Inflation

Window Length	Additive Inflataion	Multiplicative Inflation
30s	0.05	0.15

Appendix B: Code

The core components of the code necessary to reproduce all results herein is as follows. None of OpenFOAM version 2.2.1 is included, as it is freely available for download online.

```
classdef OpenFOAM<handle
    %OpenFOAM: class for an OpenFOAM run
    %   Goal is to implement a class for which MATLAB can
    %   control and run an OpenFOAM instance
    %
    %   USAGE:
    %       -initialize an OpenFOAM directory, read to be run, at 'testCase'
    %       ens1 = OpenFOAM();
    %       ens1.initFOAM('testCase');
    %       -set the T variable for the initial run
    %       ens1.T = 300*ones(36000,1);
    %       -run OpenFOAM on this case
    %       ens1.runFoam();
    %       -run more into the future
    %       ens1.ETIME = 40;
    %       ens1.runFoam();
    %       -go back and read T from time 30 (note they need to be divisors of 5 now
    %       ens1.readFoam(30)
```

APPENDIX B. CODE

```
properties
    foamLab = '/users/a/r/areagan/work/2013/data-assimilation/OpenFOAM/foamLab.sh'
    TIME=0;
    ETIME=20;
    DIR='testCase'
    FLUX=10000; % for flux BC
    BCbottom = '310'; % for fixed value BC
    BCtop = '290';
    BC = 'fixedValue';
    TURB = 'off';
    TURBMODEL = 'kEpsilon';
    T = 300*ones(36000,1);
    U = 1e-6*ones(36000,3);
    p = 1e-6*ones(36000,1);
    p_rgh = 1e-6*ones(36000,1);
    Tstep = 1;
    WRITEp = 6;
    WRITEint = 20;
    VALUE = [];
    dataVec
end % properties

methods
    function OF = OpenFOAM(varargin)
        % initialize the class
    end
    function init(OF, varargin)
        if nargin > 1
            OF.DIR = varargin{1};
        end
    end
end
```

APPENDIX B. CODE

```

end

% initialize a a clean dir
system(sprintf('%s -i -d %s',OF.foamLab,OF.DIR));
% !OF.foamLab -i -d OF.DIR
%OF.setFlux
end

function read(OF,time,varargin)
% read in a specific time variable
% first: have foamLab write it as a csv
caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',OF.I);
system(sprintf('%s -r %d -d %s',OF.foamLab,time,caseDir));
OF.T = csvread(sprintf('%s/%d/T.csv',caseDir,time));
OF.U = csvread(sprintf('%s/%d/U.csv',caseDir,time));
OF.p = csvread(sprintf('%s/%d/p.csv',caseDir,time));
OF.p_rgh = csvread(sprintf('%s/%d/p_rgh.csv',caseDir,time));
%return T
end

function slice(OF,num,varargin)
% 'num' is the slice ID
% this can be 1-8, representing the obviousness below
% the top slice Cell IDs from paraview
right = [35098 35156 35213 35268 35321 35372 35421 35468 35513 35556 35598];
oneThirty = 0:1:29;
top = 17218:58:18900;
tenThirty = 0:1:29;
left = [ 0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361];
sevenThirty = 0:1:29;
bottom = 17159:58:18841;
fourThirty = 0:1:29;

```

APPENDIX B. CODE

```

% add 1 bc matlab indexes at 1
allCellID = [right+1; oneThirty+1; top+1; tenThirty+1; left+1; sevenThir
%disp(allCellID)
cellID=allCellID(num,:);
%disp(cellID)
OF.VALUE = [mean(OF.T(cellID)) mean(OF.U(cellID,1)) mean(OF.U(cellID,2))
end
function run(OF, varargin)
% convert relevant variables to foam-readable and insert them,
% if necessary
caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',OF.I
tOutFile = sprintf('%s/%d/%s',caseDir,OF.TIME,'T.csv');
csvwrite(tOutFile,OF.T);

% run the case
system(sprintf('%s -x -d %s -t %d -h %s -g %s -e %d -l %d -w %d -c %d -b
OF.TIME = OF.ETIME;

OF.T = csvread(sprintf('%s/%d/T.csv',caseDir,OF.ETIME));
end
function makeDataVec(OF, direction)
switch direction
case 'in' % into the datavec
disp('converting the variables OF.T ... into OF.dataVec');
OF.dataVec = ones(36000*6,1); % column vector
OF.dataVec(1:6:36000*6) = OF.T;
OF.dataVec((1:6:36000*6)+1) = OF.U(:,1);
OF.dataVec((1:6:36000*6)+2) = OF.U(:,2);
OF.dataVec((1:6:36000*6)+3) = OF.U(:,3);

```

APPENDIX B. CODE

```
        OF.dataVec((1:6:36000*6)+4) = OF.p;
        OF.dataVec((1:6:36000*6)+5) = OF.p_rgh;
    case 'out' % out of dataVec into T,U,p,p_rgh
        if length(OF.dataVec) > 1 % check that something is there
            OF.T = OF.dataVec(1:6:36000*6);
            OF.U(:,1) = OF.dataVec((1:6:36000*6)+1);
            OF.U(:,2) = OF.dataVec((1:6:36000*6)+2);
            OF.U(:,3) = OF.dataVec((1:6:36000*6)+3);
            OF.p = OF.dataVec((1:6:36000*6)+4);
            OF.p_rgh = OF.dataVec((1:6:36000*6)+5);
        else
            disp('there is nothing in dataVec...did you mean to put in')
        end
    end
end
end % methods

end % classdef
```